

PROJECT REPORT

Restaurant Management System

A menu-driven Python application for ordering, billing and menu management

LANGUAGE	Python 3
APPLICATION TYPE	Console (command-line)
LIBRARIES	Python standard library only
DATE	September 2026

Table of Contents

1. Abstract	2
2. Introduction	2
3. Objectives	2
4. Scope of the Project	2
5. System Requirements	3
6. System Design	3
7. Module Description	4
8. Implementation Details	5
9. Program Flow	6
10. Sample Input and Output	6
11. Testing	7
12. Limitations	8
13. Future Enhancements	8
14. Conclusion	8

1. Abstract

The Restaurant Management System is a menu-driven Python program that automates two routine restaurant tasks: taking customer orders with bill generation, and maintaining the food menu. The program runs in the terminal, stores menu items and prices in a Python dictionary, and produces a neatly formatted bill showing the customer's details, the ordered items, quantities, amounts, and the total payable.

2. Introduction

Restaurants handle a constant flow of orders, and calculating bills by hand is slow and error-prone. A simple software system reduces calculation mistakes, speeds up billing, and makes it easy to change the menu when items or prices change.

This project implements such a system using core Python concepts: functions, dictionaries, lists, loops, conditional statements, string formatting, and user input handling. It is simple enough for beginners to follow while still covering a complete workflow from order to bill.

3. Objectives

- Record a customer's name and table number.
- Display the restaurant menu with prices.
- Let the user place an order with multiple items and quantities.
- Calculate item totals and the grand total automatically.
- Generate a formatted, readable bill.
- Allow staff to add, remove, update, and view menu items.
- Structure the code into small, reusable functions.

4. Scope of the Project

Included	Not included
Customer order taking and billing	Permanent storage (files or database)
Menu management (add, remove, update, display)	Taxes, discounts or service charges
Case-insensitive item entry	Login or user roles
Console-based interaction	Graphical user interface
	Order history or sales reports

5. System Requirements

Software requirements

Component	Requirement
Operating system	Windows, macOS or Linux
Language	Python 3.6 or higher (uses f-strings)
Editor / IDE	Any (IDLE, VS Code, PyCharm or a terminal)
External modules	None

Hardware requirements

Any computer capable of running Python. No special hardware is needed.

6. System Design

6.1 Data structures

Structure	Purpose
<code>menu</code> (dict)	Stores food items as keys and prices as values, for example <code>"Pizza": 8</code>
<code>orders</code> (list)	Stores each ordered item as <code>[item_name, quantity, item_total]</code>
<code>total</code> (int)	Running grand total of the current order

Default menu

Item	Price (\$)
Burger	4
Pizza	8
Pasta	5
Fried Rice	6
Coffee	7

6.2 Program structure

The program is divided into independent functions, each performing one task. `main_menu()` acts as the controller and calls the other functions as needed.

```
main_menu()
+-- get_customer_details()
+-- display_menu()
+-- place_order()
+-- generate_bill()
+-- manage_menu()
    +-- add_item()
    +-- remove_item()
    +-- update_price()
    +-- display_menu()
```

7. Module Description

Function	Description
<code>get_customer_details()</code>	Asks for the customer's name and table number and returns both values.
<code>display_menu()</code>	Loops through the menu dictionary and prints each item with its price in a formatted table.
<code>add_item()</code>	Takes a new food name and price, converts the name to title case, and stores it in the menu.
<code>remove_item()</code>	Removes an item from the menu if it exists; otherwise shows a "not available" message.
<code>update_price()</code>	Asks for an existing item and a new price, then updates the menu.
<code>manage_menu()</code>	Shows the menu management submenu and repeats until the user chooses Back.
<code>place_order()</code>	Repeatedly asks for an item and quantity, computes item totals, builds the order list and running total, and stops when the user enters q. Returns the orders and the total.
<code>generate_bill()</code>	Prints the customer name, table number, each item with quantity and amount, the total and a thank-you message.
<code>main_menu()</code>	Entry point. Shows the main menu and repeats until the user chooses Exit.

Main menu	Action
1	Customer Order
2	Manage Menu
3	Exit

Submenu	Action
1	Add Food Item
2	Remove Food Item
3	Update Food Item
4	Display Menu
5	Back

8. Implementation Details

Python concepts used

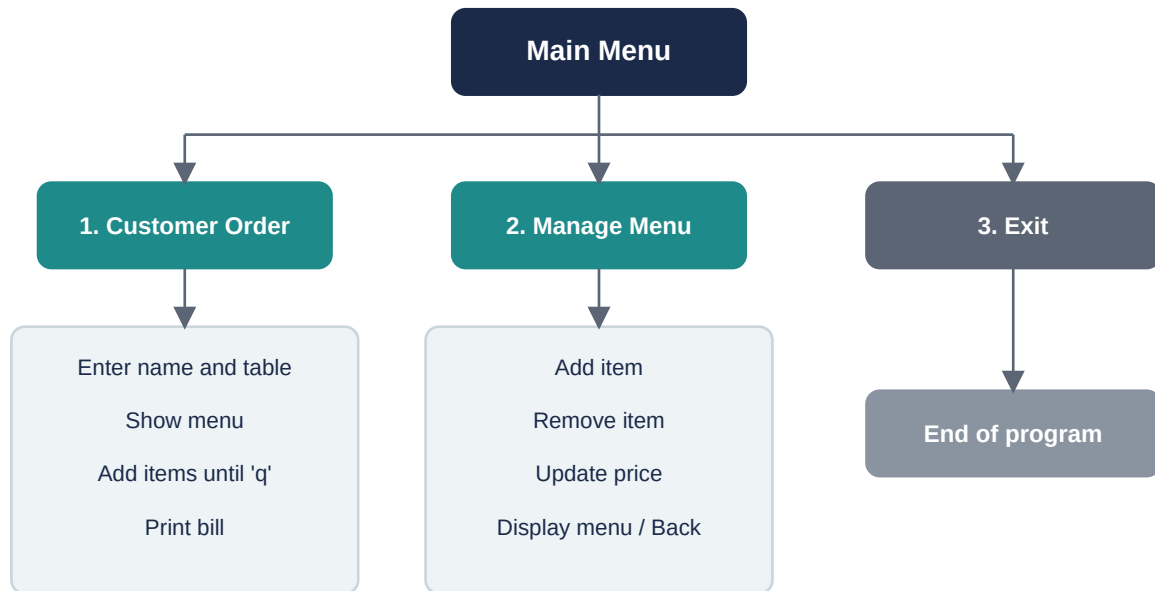
- **Functions** with parameters and return values, for example `return orders, total`.
- **Dictionaries** for the menu, updated with `menu[item] = price` and `menu.pop(item)`.
- **Lists** for holding order entries.
- **Loops:** `while True` for menus and `for` for displaying items.
- **Conditionals:** `if / elif / else` for choices and validation.
- **String methods:** `.title()` for case-insensitive matching.
- **f-strings** with alignment (for example `{item:<15}`) for column formatting.
- **Type conversion:** `int()` for prices and quantities.

Key design decisions

- Item names are converted with `.title()`, so pizza, PIZZA and Pizza are treated the same.
- The quit key q is checked after `.title()`, so it is compared as "Q".
- The `menu` dictionary is global, so every function shares the same up-to-date menu.
- Orders are created inside `place_order()` for each customer, so bills never mix.

9. Program Flow

The diagram below shows how control moves between the main menu and its options.



Order and Manage Menu both return to the Main Menu when finished

Figure 1: Overall program flow

10. Sample Input and Output

Sample order

```
Enter Customer's name: Alex
Enter Table number: 5

Enter the item from menu (or press 'q' to finish): pizza
Enter quantity: 2
2 x Pizza added to your order.
Item total: $ 16

Enter the item from menu (or press 'q' to finish): coffee
Enter quantity: 1
1 x Coffee added to your order.
Item total: $ 7

Enter the item from menu (or press 'q' to finish): q
```

Generated bill

```

=====
                YOUR BILL
=====
Customer: Alex
Table No: 5
-----
ITEM            QTY  AMOUNT
Pizza           2    $16
Coffee          1    $7
-----
TOTAL                        $23
=====
        Thank you for dining with us!
        Have a great day
=====

```

11. Testing

The test cases below were derived by tracing the program's logic for each input.

No.	Test case	Input	Expected result	Result
1	Order a valid item	Pizza, qty 2	Item total \$16 added	Pass
2	Case-insensitive name	plzZa	Recognised as Pizza	Pass
3	Item not on menu	Sushi	"The item is not available."	Pass
4	Finish an order	q	Order ends, bill is shown	Pass
5	Multiple items	Pizza x2, Coffee x1	Total shown as \$23	Pass
6	Add a menu item	Salad, price 5	Item added to the menu	Pass
7	Remove an item	Pasta	Item removed from the menu	Pass
8	Remove missing item	Sushi	"Not available" message	Pass
9	Update a price	Burger, new price 6	Price updated to \$6	Pass
10	Update missing item	Sushi	"Not available" message	Pass
11	Invalid main choice	9	"Invalid choice" message	Pass
12	Invalid submenu choice	9	"Invalid Choice" message	Pass
13	Exit the program	3	Thank-you message, program ends	Pass
14	Non-numeric quantity or price	abc	Raises ValueError and stops	Fail
15	Order with no items	q immediately	Empty bill, total \$0	Pass

Summary: 14 of 15 cases pass. The single failure (non-numeric input) is listed under limitations and future enhancements.

12. Limitations

- **No data persistence:** menu changes are lost when the program closes, as the menu lives only in memory.
- **No numeric input validation:** text or an empty value for a price or quantity raises a `ValueError` and crashes the program.
- **Negative or zero values are accepted** for prices and quantities.
- **Whole-number prices only:** decimals are not supported because `int()` is used.
- **No order history:** bills are printed but not saved.
- **No taxes or discounts**, and **no login** or role separation.
- **Minor typo:** the message "not avaiable" in `remove_item()` should read "not available".

13. Future Enhancements

- Add `try / except` blocks to validate all numeric input.
- Reject negative or zero prices and quantities.
- Support decimal prices using `float` or the `decimal` module.
- Save the menu and sales records in JSON, CSV or a database such as SQLite.
- Add tax, service charge and discount calculations.
- Save each bill as a text or PDF file.
- Add a daily sales report and a staff login system.
- Build a graphical interface with Tkinter, or a web version.

14. Conclusion

The Restaurant Management System meets its objectives. It gives restaurant staff a simple way to manage the menu, take orders, and produce accurate bills instantly. The code is organised into clear, single-purpose functions, which makes it easy to read, test, and extend.

While the current version is limited to in-memory storage and basic validation, its modular design means features such as file storage, input validation and taxes can be added without rewriting the existing logic. The project is a good demonstration of core Python concepts applied to a practical, real-world problem.